

Omnitech

Testing policy doc

This is the testing policy document for team omnitech which details the structure of the project. The purpose of this document is to ensure that coding standards are kept and that all code that reaches our production is tested and must remain easily maintainable.

Testing procedures

Our project's testing strategy is structured as follows:

1. Automated tests

All tasks, including unit and integration tests, are automated to ensure the code is consistently tested and meets quality standards.

2. Continuous feedback through CI

Testing is integrated into our CI to ensure immediate feedback after every pull request.

Testing tools and justification

The following frameworks and tools were used during tests:

1. Jest:

Jest was used because it is native to the language used in our backend (typescript). It also allows for parallel execution of tests which allows for tests to be evaluated faster. Jest was used for our unit tests because it allows for built-in mocking.

2. Supertest:

Supertest was used by integration because it provides the network simulation layer needed for HTTP requests. Supertest is a complimentary pairing with jest for node.js (typescript) backends.

3. Continuous Integration (CI) github actions:

Our testing pipeline is implemented using GitHub Actions to automate the execution of unit and integration tests. GitHub Actions was chosen because it provides native integration with the repository, making it straightforward to configure and maintain. This simplifies the continuous integration (CI) process by reducing the need for additional third-party tools while ensuring that tests are run automatically on every change, helping to maintain code quality and reliability.

Test procedure

1. Unit tests:

Two levels of unit testing were implemented in the project: controller-level tests and service-level tests. Dependencies between layers, such as the Prisma data access layer, were mocked to isolate the components under test. This approach ensured that each backend endpoint and service could be tested independently using Jest, allowing individual units to be validated without relying on external dependencies. As a result, tests execute quickly, provide reliable feedback, and help verify the correctness of the application's business logic.

2. Integration tests:

Integration tests were implemented by making a test DB to prevent any mocking of services. Supertest was used for the simulation of endpoint calls and checking whether the endpoint interacts as expected with the db and other functions, also to check whether the different levels are communicating as expected.

3. End-to-End (E2E) tests:

For end-to-end testing, UI Automator with JUnit 4 was used to simulate user interactions by stepping through the application. This was achieved by assigning tags to the required input fields and user interface elements, allowing the automation framework to accurately identify and interact with components on the screen.

These tests validate the complete application workflow by exercising both the frontend and backend. The automated tests interact with the user interface as a real user would while making actual API calls to the backend, ensuring that the application's functionality operates correctly from end to end. This approach also verifies that user interface elements remain functional and that integration between the frontend and backend is working as expected.

4. Overall procedure:

As new units of functionality were implemented, the team was required to write and maintain corresponding automated tests. Before a pull request (PR) could be approved, all tests had to pass and the project was required to maintain a minimum test coverage of 80%.

This process was enforced automatically through GitHub Actions, which executed the test suite and verified code coverage for every pull request. By integrating these checks into the continuous integration (CI) pipeline, only well-tested, maintainable code that met the project's quality standards could be merged into the shared branches.

Continuous Deployment (CD)

Deployment Testing Policy

Purpose

This deployment pipeline ensures that every production deployment is automatically validated and, where necessary, rolled back to the last known healthy version. The goal is to minimize production downtime while ensuring that only healthy application versions remain deployed.

Deployment Trigger

The deployment workflow is automatically triggered on every push to the `main` branch.

This establishes a continuous deployment (CD) process where:

1. Changes are merged into `main`.
2. A container image is built.
3. The image is deployed to production.
4. The deployment is validated through automated health checks.

Deployment Process

1. Source Checkout

The workflow checks out the repository to provide access to the application source code and the Docker build context located in `./backend`.

2. Azure Authentication

The GitHub Actions runner authenticates with Azure using a service principal stored in the `AZURE_CREDENTIALS` secret. This authorises all subsequent Azure CLI operations.

3. Azure Container Registry (ACR) Authentication

Docker authenticates with Azure Container Registry using repository secrets so that images can be pushed securely.

4. Container Image Build

A Docker image is built from the `./backend` directory.

Each image is tagged using the Git commit SHA that triggered the workflow, providing:

- unique image versions
- deployment traceability
- simplified rollback

Image format:

```
<REGISTRY>/<IMAGE_NAME>:<COMMIT_SHA>
```

5. Push Image to Registry

The newly built image is pushed to Azure Container Registry, making it available for deployment.

6. Capture Current Production Image

Before deployment begins, the workflow retrieves the currently deployed container image from Azure App Service using:

```
az webapp sitecontainers list
```

The image reference is stored as `previous_image` and serves as the rollback target if deployment validation fails.

7. Deploy New Image

The App Service Site Container named `main` is updated to reference the newly built image.

Azure Container Registry credentials are supplied to allow the App Service to pull the image.

8. Restart Application

The Azure App Service is restarted to ensure the updated container image is loaded and running.

9. Automated Health Verification

Following deployment, the application health endpoint is monitored:

```
https://<app>.azurewebsites.net/health
```

Health check behaviour:

- Poll every 10 seconds.
- Maximum of 30 attempts.
- Total timeout of approximately 5 minutes.
- Deployment succeeds immediately upon receiving an HTTP 200 response.
- Failure to receive a successful response causes the workflow to fail.

10. Automatic Rollback

If any deployment-stage step fails, the workflow automatically attempts to restore the previously deployed container image.(image tag pinning)

Rollback actions include:

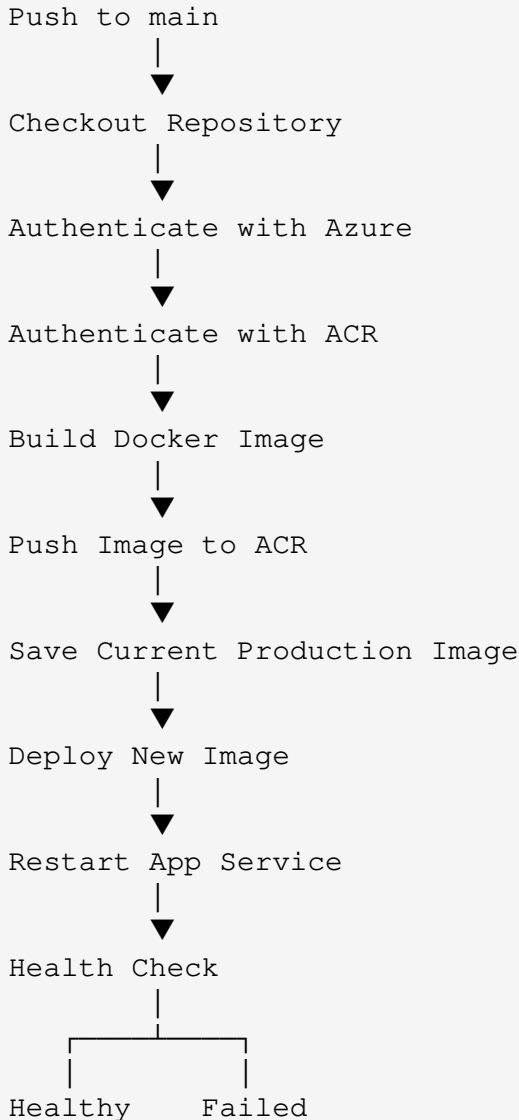
1. Redeploy the saved `previous_image`.
2. Restart the Azure App Service.
3. Restore the last known healthy deployment.

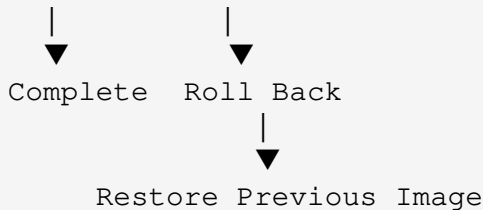
This rollback is executed automatically using the GitHub Actions `if: failure()` condition.

Environment Variables

Variable	Purpose
IMAGE_TAG	Git commit SHA used as the Docker image tag
REGISTRY	Azure Container Registry login server
IMAGE_NAME	Repository image name
SITE_CONTAINER_NAME	Name of the Site Container (main) updated during deployment

Deployment Validation Flow





Current Considerations

The current workflow is reliable but has several considerations that should be documented.

Initial Deployment

During the first deployment, there may be no existing production image available.

In this scenario, `previous_image` will be empty, preventing a successful rollback. The workflow should therefore skip rollback when no previous deployment exists.

Rollback Verification

Following a rollback, the workflow currently assumes the restoration succeeded.

For improved reliability, a second health check should be performed after rollback to confirm that the application has returned to a healthy state.

Rollback Trigger Scope

The rollback step uses `if: failure()`, meaning it executes whenever any previous workflow step fails.

This includes failures unrelated to deployment (for example authentication or image push failures). Although generally harmless, rollback is only meaningful if a deployment was actually attempted.

Authentication Improvements

Azure Container Registry currently uses stored credentials for authentication.

Where possible, OpenID Connect (OIDC)-based authentication should be adopted to eliminate long-lived registry credentials and improve overall security.